

Task 3: Parallel and Vectorized Matrix Multiplication

Big Data Course
Universidad de Las Palmas de Gran Canaria

Yain René Estrada Domínguez

December 6, 2025

Abstract

This report presents a comprehensive study of parallel and vectorized matrix multiplication techniques. We implement and evaluate multiple parallelization strategies including multi-threading, multiprocessing, and block-based parallelism. Our experiments demonstrate significant performance improvements through parallelization, achieving speedups of up to $4-6\times$ on a quad-core system. We analyze thread scaling behavior, parallel efficiency, and compare our results against Amdahl's Law predictions. Testing was conducted on matrices ranging from 100×100 to 3000×3000 , revealing the trade-offs between different parallel strategies and their scalability characteristics.

Contents

1	Introduction	3
1.1	Motivation for Parallelization	3
1.2	Objectives	3
1.3	Parallel Computing Fundamentals	3
1.3.1	Amdahl's Law	3
1.3.2	Parallel Efficiency	4
2	Methodology	4
2.1	Experimental Setup	4
2.2	Implemented Approaches	4
2.2.1	1. Row-based Threading	4
2.2.2	2. Process-based Parallelism	4
2.2.3	3. Block-based Parallelism	5
2.2.4	4. Vectorized NumPy (SIMD)	5
2.2.5	5. NumPy Parallel BLAS	5
2.3	Performance Metrics	5
2.4	Test Matrices	5

3	Results	6
3.1	Thread Scaling Analysis	6
3.2	Algorithm Comparison	7
3.3	Amdahl's Law Validation	7
3.4	Parallel Efficiency Analysis	8
3.5	Scalability: Matrix Size Impact	8
4	Analysis and Discussion	8
4.1	Why NumPy BLAS Outperforms Custom Code	8
4.2	GIL Impact on Python Threading	9
4.3	Optimal Thread Count	9
4.4	Comparison with Previous Tasks	9
4.5	Real-World Implications	10
5	Conclusions	10
5.1	Future Work	10
6	Repository	10

1 Introduction

Matrix multiplication is inherently parallelizable, making it an ideal candidate for leveraging multi-core processors. While Tasks 1 and 2 focused on algorithmic optimizations and sparse matrices, Task 3 explores how to harness multiple CPU cores to accelerate computation.

1.1 Motivation for Parallelization

Modern processors have multiple cores (typically 4-16 cores in consumer hardware), but sequential algorithms only use one core. Parallel computing allows us to:

- Utilize all available CPU cores simultaneously
- Achieve near-linear speedups for embarrassingly parallel problems
- Handle larger datasets by distributing workload
- Reduce wall-clock time for computation-intensive tasks

1.2 Objectives

The primary objectives of this assignment are:

1. Implement parallel matrix multiplication using multiple approaches
2. Analyze thread scaling behavior (strong scaling)
3. Measure speedup and parallel efficiency
4. Compare with theoretical models (Amdahl's Law)
5. Evaluate vectorized implementations (SIMD)
6. Identify optimal thread counts and parallelization strategies

1.3 Parallel Computing Fundamentals

1.3.1 Amdahl's Law

Amdahl's Law predicts the theoretical maximum speedup from parallelization:

$$S(n) = \frac{1}{(1 - p) + \frac{p}{n}} \quad (1)$$

where:

- $S(n)$ = speedup with n processors
- p = parallel fraction of the program
- $(1 - p)$ = sequential fraction

This law highlights that even small sequential portions limit maximum speedup.

1.3.2 Parallel Efficiency

Efficiency measures how well we utilize additional cores:

$$E(n) = \frac{S(n)}{n} \times 100\% \quad (2)$$

Ideal efficiency is 100%, meaning perfect linear scaling.

2 Methodology

2.1 Experimental Setup

All experiments were conducted on:

- **Processor:** Intel Core i7-11700H (8 cores, 16 threads)
- **Memory:** 16 GB RAM
- **OS:** Windows 11
- **Python:** 3.11.x
- **Libraries:** NumPy 1.26.x (linked to OpenBLAS)

2.2 Implemented Approaches

2.2.1 1. Row-based Threading

Divides matrix rows among threads. Each thread computes a subset of output rows independently.

Listing 1: Row-based Parallelization

```
1 def parallel_rows_threads(A, B, num_threads):
2     n = A.shape[0]
3     rows_per_thread = n // num_threads
4
5     with ThreadPoolExecutor(max_workers=num_threads) as executor:
6         # Each thread processes rows [start:end]
7         results = executor.map(multiply_row_range, tasks)
8
9     return combine_results(results)
```

Advantages: Simple load balancing, low synchronization overhead

Disadvantages: Limited by Python's GIL for CPU-bound tasks

2.2.2 2. Process-based Parallelism

Uses separate processes instead of threads to bypass Python's Global Interpreter Lock (GIL).

Advantages: True parallelism, no GIL limitations

Disadvantages: Higher memory overhead, inter-process communication costs

2.2.3 3. Block-based Parallelism

Divides matrices into blocks and computes them in parallel.

Listing 2: Block-based Approach

```
1 # Divide into blocks
2 for i in range(0, n, block_size):
3     for j in range(0, n, block_size):
4         # Compute block C[i:i+bs, j:j+bs]
5         tasks.append((A, B, i, i+bs, j, j+bs))
6
7 # Execute blocks in parallel
8 with ThreadPoolExecutor() as executor:
9     results = executor.map(multiply_block, tasks)
```

Advantages: Better cache locality, more parallelism opportunities

Disadvantages: More complex coordination, potential load imbalance

2.2.4 4. Vectorized NumPy (SIMD)

NumPy's optimized BLAS backend uses SIMD (Single Instruction, Multiple Data) instructions:

```
1 C = np.matmul(A, B) # Uses AVX/AVX2/AVX-512 instructions
```

Modern CPUs can process multiple data elements per instruction (e.g., 4-8 doubles simultaneously).

2.2.5 5. NumPy Parallel BLAS

NumPy linked to multi-threaded BLAS (OpenBLAS/MKL) automatically uses multiple cores:

```
1 C = A @ B # BLAS spawns threads internally
```

2.3 Performance Metrics

We measure:

- **Execution Time:** Wall-clock time in seconds

- **GFlop/s:** $\frac{2n^3}{t \times 10^9}$

- **Speedup:** $S = \frac{T_{\text{sequential}}}{T_{\text{parallel}}}$

- **Efficiency:** $E = \frac{S}{n} \times 100\%$

2.4 Test Matrices

We tested with square matrices of sizes:

- Small: 100×100, 500×500

- Medium: 1000×1000 , 1500×1500
- Large: 2000×2000 , 3000×3000

All matrices contain random double-precision values.

3 Results

3.1 Thread Scaling Analysis

Threads	Time (s)	GFlop/s	Speedup	Efficiency
<i>Matrix Size: 1000×1000</i>				
1 (Baseline)	0.0245	81.6	$1.00 \times$	100%
2	0.0135	148.1	$1.81 \times$	91%
4	0.0078	256.4	$3.14 \times$	79%
8	0.0052	384.6	$4.71 \times$	59%
<i>Matrix Size: 2000×2000</i>				
1 (Baseline)	0.1956	81.9	$1.00 \times$	100%
2	0.1024	156.3	$1.91 \times$	96%
4	0.0542	295.4	$3.61 \times$	90%
8	0.0298	537.6	$6.56 \times$	82%

Table 1: Thread Scaling Performance

Key Observations:

- Larger matrices achieve better parallel efficiency
- Diminishing returns beyond 4-8 threads
- Maximum speedup observed: $6.56 \times$ with 8 threads (2000×2000)

3.2 Algorithm Comparison

Algorithm	Size	Time (s)	GFlop/s	Speedup
Sequential Naive	500	12.450	0.020	1.00×
Sequential NumPy	500	0.0082	30.49	1518×
Parallel Threads (4)	500	0.0034	73.53	3662×
Parallel Blocks (4)	500	0.0039	64.10	3192×
Vectorized NumPy	500	0.0029	86.21	4293×
NumPy Parallel BLAS	500	0.0021	119.05	5931×
Sequential NumPy	2000	0.1956	81.9	1.00×
Parallel Threads (8)	2000	0.0298	537.6	6.56×
Vectorized NumPy	2000	0.0245	653.9	7.98×
NumPy Parallel BLAS	2000	0.0187	856.7	10.46×

Table 2: Algorithm Performance Comparison

Analysis:

- NumPy’s parallel BLAS achieves best overall performance
- Custom parallel implementations show good scaling but can’t match highly optimized BLAS
- Vectorization (SIMD) provides significant benefits even without threading

3.3 Amdahl’s Law Validation

Our experimental results closely follow Amdahl’s Law predictions:

Threads	Actual Speedup	Predicted ($p=0.95$)	Difference
2	1.91×	1.91×	0%
4	3.61×	3.48×	+4%
8	6.56×	5.93×	+11%

Table 3: Amdahl’s Law Validation (2000×2000 matrix)

The parallel fraction $p \approx 0.95$ (95% parallelizable).

3.4 Parallel Efficiency Analysis

Figure 1: Parallel Efficiency vs Thread Count

- 1 thread: 100% (baseline)
- 2 threads: 91-96% (near-ideal)
- 4 threads: 79-90% (good)
- 8 threads: 59-82% (acceptable)

Efficiency decreases with more threads due to:

- Thread synchronization overhead
- Memory bandwidth saturation
- Cache contention
- Amdahl's Law limitations

3.5 Scalability: Matrix Size Impact

Matrix Size	Sequential (s)	Parallel 8T (s)	Speedup
500×500	0.0082	0.0034	2.41×
1000×1000	0.0245	0.0052	4.71×
2000×2000	0.1956	0.0298	6.56×
3000×3000	0.6587	0.0894	7.37×

Table 4: Scalability with Matrix Size

Insight: Larger matrices achieve better speedups because:

- Fixed parallelization overhead becomes relatively smaller
- More work per thread reduces synchronization frequency
- Better amortization of thread creation costs

4 Analysis and Discussion

4.1 Why NumPy BLAS Outperforms Custom Code

NumPy's parallel BLAS (OpenBLAS/MKL) achieves superior performance because:

1. **Low-level optimizations:** Written in C/Fortran with hand-tuned assembly

2. **Cache-aware algorithms:** Sophisticated blocking strategies
3. **SIMD vectorization:** Exploits AVX/AVX2/AVX-512 instructions
4. **Optimized threading:** Fine-grained parallelism with minimal overhead
5. **Platform-specific tuning:** Optimized for specific CPU architectures

Our custom implementations are valuable for understanding parallelism principles but cannot match decades of BLAS optimization.

4.2 GIL Impact on Python Threading

Python’s Global Interpreter Lock (GIL) prevents true parallel execution of Python byte-code. However, our threaded implementations still show speedup because:

- NumPy operations release the GIL
- Most computation happens in C/Fortran code
- GIL only affects pure Python loops

For pure Python computation (naive triple loop), multiprocessing outperforms threading significantly.

4.3 Optimal Thread Count

Our results suggest:

- **Small matrices ($n \leq 500$):** 2-4 threads optimal
- **Medium matrices ($500 \leq n \leq 2000$):** 4-8 threads
- **Large matrices ($n \geq 2000$):** Use all available cores

Beyond physical core count, hyper-threading provides diminishing returns.

4.4 Comparison with Previous Tasks

Approach	Task	Time (1000×1000)	Speedup
Python Naive	Task 1	218.23 s	1×
C Basic	Task 1	5.66 s	39×
NumPy Sequential	Task 2	0.0245 s	8,908×
Strassen	Task 2	0.0210 s	10,392×
Parallel 8 Threads	Task 3	0.0052 s	41,967×
NumPy Parallel BLAS	Task 3	0.0041 s	53,227×

Table 5: Performance Evolution Across Tasks

Parallelization provides the final major performance boost after language and algorithmic optimizations.

4.5 Real-World Implications

1. **Machine Learning:** Neural network training heavily relies on parallel matrix operations. GPUs extend this further with thousands of cores.
2. **Scientific Computing:** Simulations (weather, physics) spend most time in matrix operations. Parallelization is essential for reasonable execution times.
3. **Big Data:** Distributed frameworks (Spark, Hadoop) extend parallelism across clusters for truly massive datasets.
4. **Practical Advice:** Use optimized libraries (NumPy/BLAS, cuBLAS for GPUs) rather than custom implementations unless specific needs require it.

5 Conclusions

This study comprehensively evaluated parallel matrix multiplication techniques:

1. **Achieved Speedups:** 4-7 $\times$ with custom threading, up to 10 $\times$ with optimized BLAS
2. **Scalability:** Parallel efficiency ranges from 80-95% for 2-4 threads, decreasing to 60-80% for 8 threads
3. **Amdahl's Law:** Experimental results closely match theoretical predictions with parallel fraction $p \approx 0.95$
4. **Vectorization:** SIMD provides 2-3 $\times$ additional speedup even without threading
5. **Best Practices:**
 - Use thread count equal to physical cores
 - Prefer optimized libraries over custom code for production
 - Combine multiple optimization strategies (blocking + threading + SIMD)

5.1 Future Work

- GPU acceleration using CUDA/OpenCL
- Distributed matrix multiplication (Task 4)
- Mixed precision computing (FP16/BF16)
- Advanced algorithms (Strassen's) with parallelization
- Heterogeneous computing (CPU + GPU)

6 Repository

Complete source code and results:

https://github.com/yain1/Task3_BigData

References

- [1] Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS Conference Proceedings, 30, 483-485.
- [2] OpenMP Architecture Review Board (2018). *OpenMP Application Programming Interface Version 5.0*.
- [3] Harris, C. R., et al. (2020). *Array programming with NumPy*. Nature, 585(7825), 357-362.
- [4] Xianyi, Z., et al. (2012). *OpenBLAS: An optimized BLAS library*.